```
/* كورة الصحراء — لعبة كورة قدم ثري دي أونلاين */
(() => {
'use strict';
const $ = (id) => document.getElementById(id);

/* ================== ثوابت ================== */
const FW = 44, FL = 70, HW = FW / 2, HL = FL / 2;
const GW = 9, GH = 3.2, ND = 2.2, BR = 0.36;
const BX = HW + 2, BZ = HL + 3.2;
const MATCH = 180, NPT = 5, NP = 10;
const PFX = 'korasahra26-';
const PUB_ROOMS = ['pub1', 'pub2', 'pub3'];
const TEAM_NAME = ['الأحمر', 'الأزرق'];
const TEAM_CSS = ['#3d8bff', '#ff4b45'];
const TEAM_COL = [0x2f7fff, 0xe53935];
const GK_COL = [0x22c06d, 0xffb300];
const NUMS = [1, 4, 5, 8, 11];
const FORM = [[0, -HL + 1.4], [-9, -21], [9, -21], [-8, -8], [6, -4]];
const HUMAN_ORDER = [4, 3, 1, 2];
const BOT_NAMES = ['حمد', 'عبدالله', 'مهند', 'سلطان', 'عزام', 'فيصل', 'يزيد', 'راكان'];
const SKINS = [0xf1c27d, 0xe0ac69, 0xc68642, 0x8d5524, 0xffdbac];
const HAIRS = [0x2b1a0e, 0x111111, 0x6b3e1d, 0xc0541b, 0x3a2414, 0xe6c15a];

const clamp = (v, a, b) => (v < a ? a : v > b ? b : v);
const rnd = (a, b) => a + Math.random() * (b - a);
const sgn = (t) => (t === 0 ? 1 : -1);
const dist = (a, b) => Math.hypot(a.x - b.x, a.z - b.z);
const r2 = (v) => Math.round(v * 100) / 100;
function lerpAng(a, b, t) {
  let d = (b - a) % (Math.PI * 2);
  if (d > Math.PI) d -= Math.PI * 2;
  if (d < -Math.PI) d += Math.PI * 2;
  return a + d * t;
}
const BN = BOT_NAMES.slice().sort(() => Math.random() - 0.5);
const botName = (i) => BN[i % BN.length];
const zeroInput = () => ({ mx: 0, mz: 0, sp: 0, sh: 0, pa: 0, sl: 0, ca: 0 });

/* ================== الأصوات ================== */
const SND = {
  ctx: null, noise: null, master: null,
  init() {
    if (this.ctx) { if (this.ctx.state === 'suspended') this.ctx.resume(); return
```

```
    try {
      const C = window.AudioContext || window.webkitAudioContext;
      this.ctx = new C();
      this.master = this.ctx.createGain(); this.master.gain.value = 0.7; this.mas
      const len = this.ctx.sampleRate * 2, buf = this.ctx.createBuffer(1, len, th
      for (let i = 0; i < len; i++) d[i] = Math.random() * 2 - 1;
      this.noise = buf;
    } catch (e) { this.ctx = null; }
  },
  env(node, t, a, peak, dur) {
    node.gain.setValueAtTime(0.0001, t);
    node.gain.exponentialRampToValueAtTime(peak, t + a);
    node.gain.exponentialRampToValueAtTime(0.0001, t + dur);
  },
  tone(f1, f2, dur, vol, type) {
    if (!this.ctx) return;
    const t = this.ctx.currentTime, o = this.ctx.createOscillator(), g = this.ctx
    o.type = type || 'sine'; o.frequency.setValueAtTime(f1, t); o.frequency.expon
    this.env(g, t, 0.005, vol, dur); o.connect(g); g.connect(this.master); o.star
  },
  burst(freq, q, dur, vol, start) {
    if (!this.ctx) return;
    const t = this.ctx.currentTime + (start || 0), s = this.ctx.createBufferSourc
    s.buffer = this.noise; f.type = 'bandpass'; f.frequency.value = freq; f.Q.val
    this.env(g, t, 0.01, vol, dur); s.connect(f); f.connect(g); g.connect(this.ma
  },
  kick(v) { v = clamp(v || 0.4, 0.1, 1); this.tone(160, 45, 0.14, 0.5 * v + 0.1);
  thud() { this.tone(90, 30, 0.25, 0.6); this.burst(400, 0.8, 0.2, 0.3); },
  whoosh() { this.burst(900, 0.6, 0.3, 0.18); },
  pop() { this.tone(600, 1200, 0.12, 0.25, 'triangle'); },
  whistle(long) {
    if (!this.ctx) return;
    const t = this.ctx.currentTime, d = long ? 1.1 : 0.45, o = this.ctx.createOsc
    o.frequency.value = 2600; l.frequency.value = 28; lg.gain.value = 160; l.conn
    g.gain.setValueAtTime(0.0001, t); g.gain.exponentialRampToValueAtTime(0.22, t
    o.connect(g); g.connect(this.master); o.start(t); l.start(t); o.stop(t + d +
  },
  goal() {
    this.burst(1100, 0.4, 2.4, 0.5); this.burst(600, 0.5, 2.0, 0.35, 0.1);
    [523, 659, 784, 1046].forEach((f, i) => setTimeout(() => this.tone(f, f, 0.3,
  },
};

/* ================= حالة المباراة ================= */
let S = null;
let myId = -1;
```

```
let running = false;
const LI = zeroInput(); // مدخلات اللاعب المحلي

function newPlayer(id) {
  const team = id < NPT ? 0 : 1, slot = id % NPT;
  return {
    id, team, slot, gk: slot === 0, bot: true, name: botName(id), num: NUMS[slot]
    x: 0, z: 0, vx: 0, vz: 0, rot: team === 0 ? 0 : Math.PI,
    st: 0, stT: 0, stam: 1, slideCd: 0, noTouch: 0, protect: 0, call: 0, charge:
    think: Math.random() * 0.3, passTo: -1, passDelay: 0, holdT: 0,
    input: zeroInput(), prev: null, prevSh: 0, conn: null,
    tx: 0, tz: 0, trot: 0,
  };
}
function newState() {
  const s = {
    players: [], ball: { x: 0, y: BR, z: 0, vx: 0, vy: 0, vz: 0, owner: -1, last:
    score: [0, 0], time: MATCH, phase: 'kickoff', phaseT: 2.4, kickTeam: Math.ran
    ev: [], rosterVer: 0, autoRestart: false, goalTeam: -1,
  };
  for (let i = 0; i < NP; i++) s.players.push(newPlayer(i));
  return s;
}
function placeKickoff() {
  for (const p of S.players) {
    const f = FORM[p.slot], k = sgn(p.team);
    p.x = f[0]; p.z = f[1] * k; p.vx = p.vz = 0; p.st = 0; p.stT = 0; p.charge =
    p.rot = p.team === 0 ? 0 : Math.PI;
    if (p.team === S.kickTeam && p.slot === 4) { p.x = 0; p.z = -1.1 * k; }
  }
  const b = S.ball;
  Object.assign(b, { x: 0, y: BR, z: 0, vx: 0, vy: 0, vz: 0, owner: -1, last: -1
  S.phase = 'kickoff'; S.phaseT = 2.2; S.goalTeam = -1;
}
function restartMatch() {
  S.score = [0, 0]; S.time = MATCH; S.kickTeam = Math.random() < 0.5 ? 0 : 1;
  placeKickoff(); S.ev.push(['r']);
}
function endMatch() { S.phase = 'end'; S.phaseT = 12; S.ball.owner = -1; S.ev.pus

/* ================== المحاكاة ================== */
function step(dt) {
  const s = S;
  if (s.phase === 'end') {
    s.phaseT -= dt;
    if (s.autoRestart && s.phaseT <= 0) restartMatch();
```

```
    for (const p of s.players) { p.vx *= 0.9; p.vz *= 0.9; }
    return;
  }
  if (s.phase === 'kickoff') { s.phaseT -= dt; if (s.phaseT <= 0) { s.phase = 'pl
  else if (s.phase === 'goal') { s.phaseT -= dt; if (s.phaseT <= 0) { if (s.time
  else if (s.phase === 'play') { s.time -= dt; if (s.time <= 0) { s.time = 0; s.e
  const frozen = s.phase === 'kickoff';
  if (!frozen) for (const p of s.players) if (p.bot) botThink(p, dt);
  for (const p of s.players) updatePlayer(p, dt, frozen);
  separate();
  if (s.phase === 'play') steals(dt);
  updateBall(dt);
}

function updatePlayer(p, dt, frozen) {
  p.slideCd = Math.max(0, p.slideCd - dt); p.noTouch = Math.max(0, p.noTouch - dt
  p.protect = Math.max(0, p.protect - dt); p.call = Math.max(0, p.call - dt);
  const inp = p.input;
  if (!p.bot) {
    if (!p.prev) p.prev = { pa: inp.pa, sl: inp.sl, ca: inp.ca };
    if (!frozen && S.phase !== 'end') {
      if (inp.pa !== p.prev.pa) actPass(p);
      if (inp.sl !== p.prev.sl) actSlide(p);
      if (inp.ca !== p.prev.ca) actCall(p);
      if (inp.sh) p.charge = Math.min(1, p.charge + dt * 1.25);
      else if (p.prevSh) { actShoot(p, 0.25 + 0.75 * p.charge); p.charge = 0; }
      else p.charge = 0;
    }
    p.prev.pa = inp.pa; p.prev.sl = inp.sl; p.prev.ca = inp.ca; p.prevSh = inp.sh
  }
  if (p.st === 2) { // طايح
    p.stT -= dt; const f = Math.exp(-4 * dt); p.vx *= f; p.vz *= f;
    if (p.stT <= 0) p.st = 0;
  } else if (p.st === 1) { // زحلقة
    p.stT -= dt; const f = Math.exp(-2.4 * dt); p.vx *= f; p.vz *= f;
    slideHits(p);
    if (p.stT <= 0) p.st = 0;
  } else if (frozen) {
    p.vx = p.vz = 0;
  } else {
    let mx = inp.mx, mz = inp.mz, m = Math.hypot(mx, mz);
    if (m > 1) { mx /= m; mz /= m; m = 1; }
    const own = S.ball.owner === p.id;
    const sprint = inp.sp && p.stam > 0.04 && m > 0.2;
    let sp = (sprint ? 11.2 : 7.4) * (own ? 0.93 : 1) * (p.gk && p.bot ? 1.1 : 1)
    if (p.st === 3) { // اندفاع بالكورة
```

```
      p.stT -= dt; if (p.stT <= 0) p.st = 0;
      if (m > 0.2) p.rot = lerpAng(p.rot, Math.atan2(mx, mz), 1 - Math.exp(-5 * d
      p.vx = Math.sin(p.rot) * 15.5; p.vz = Math.cos(p.rot) * 15.5;
    } else {
      const a = 1 - Math.exp(-(m > 0.05 ? 9 : 7) * dt);
      p.vx += (mx * sp - p.vx) * a; p.vz += (mz * sp - p.vz) * a;
      if (m > 0.12) p.rot = lerpAng(p.rot, Math.atan2(mx, mz), 1 - Math.exp(-14 *
    }
    if (sprint) p.stam = Math.max(0, p.stam - dt * 0.28); else p.stam = Math.min(
  }
  p.x += p.vx * dt; p.z += p.vz * dt;
  p.x = clamp(p.x, -BX + 0.6, BX - 0.6); p.z = clamp(p.z, -BZ + 0.6, BZ - 0.6);
  // لا يدخل الشباك
  const ax = Math.abs(p.x), az = Math.abs(p.z), gx = GW / 2 + 0.6;
  if (az > HL && ax < gx) {
    if (az - HL < gx - ax) p.z = Math.sign(p.z) * HL; else p.x = Math.sign(p.x ||
  }
}

function separate() {
  const P = S.players;
  for (let i = 0; i < NP; i++) {
    const a = P[i]; if (a.st === 1 || a.st === 2) continue;
    for (let j = i + 1; j < NP; j++) {
      const b = P[j]; if (b.st === 1 || b.st === 2) continue;
      const dx = b.x - a.x, dz = b.z - a.z, d = Math.hypot(dx, dz);
      if (d < 0.85 && d > 0.0001) {
        const push = (0.85 - d) / 2, nx = dx / d, nz = dz / d;
        a.x -= nx * push; a.z -= nz * push; b.x += nx * push; b.z += nz * push;
      }
    }
  }
}

function steals(dt) {
  const b = S.ball; if (b.owner < 0) return;
  const o = S.players[b.owner]; if (o.gk || o.protect > 0) return;
  for (const q of S.players) {
    if (q.team === o.team || q.noTouch > 0 || !(q.st === 0 || q.st === 3)) contin
    if (Math.hypot(q.x - b.x, q.z - b.z) < 0.9 && Math.random() < dt * 1.6) {
      b.owner = q.id; b.last = q.id; q.protect = 0.6; q.holdT = 0; o.noTouch = 0.
      S.ev.push(['k', 0.2]); return;
    }
  }
}
```

```
function inOwnBox(p) {
  const gz = -HL * sgn(p.team);
  return Math.abs(p.x) < 10.5 && Math.abs(p.z - gz) < 9.5;
}

function updateBall(dt) {
  const b = S.ball, px = b.x, py = b.y, pz = b.z;
  if (b.owner >= 0) {
    const o = S.players[b.owner];
    if (o.st === 1 || o.st === 2) b.owner = -1;
    else {
      const hold = o.gk && inOwnBox(o);
      const d = hold ? 0.55 : 0.8;
      b.x = o.x + Math.sin(o.rot) * d; b.z = o.z + Math.cos(o.rot) * d; b.y = hol
      b.vx = o.vx; b.vz = o.vz; b.vy = 0;
    }
  }
  if (b.owner < 0) {
    b.vy -= 22 * dt;
    b.x += b.vx * dt; b.y += b.vy * dt; b.z += b.vz * dt;
    if (b.y < BR) { b.y = BR; if (b.vy < -3) { b.vy = -b.vy * 0.45; } else b.vy =
    const f = Math.exp((b.y <= BR + 0.01 ? -0.85 : -0.12) * dt);
    b.vx *= f; b.vz *= f;
  }
  // الحواجز
  if (b.x > BX - BR) { b.x = BX - BR; b.vx = -Math.abs(b.vx) * 0.6; }
  if (b.x < -BX + BR) { b.x = -BX + BR; b.vx = Math.abs(b.vx) * 0.6; }
  if (b.z > BZ - BR) { b.z = BZ - BR; b.vz = -Math.abs(b.vz) * 0.6; }
  if (b.z < -BZ + BR) { b.z = -BZ + BR; b.vz = Math.abs(b.vz) * 0.6; }
  // المرمى
  for (const side of [1, -1]) {
    const L = HL * side, zp = pz * side, zn = b.z * side;
    if (zp <= HL && zn > HL) {
      if (Math.abs(b.x) < GW / 2 - BR * 0.5 && b.y < GH - BR * 0.3) {
        if (S.phase === 'play') scoreGoal(side > 0 ? 0 : 1);
        if (b.owner >= 0) { b.owner = -1; }
      } else if (Math.abs(b.x) < GW / 2 + BR && b.y < GH + BR) { // ثم أو العارضة
        b.z = L - side * BR; b.vz = -b.vz * 0.6; b.owner = -1; S.ev.push(['k', 0.
      }
    }
    const inDepth = zn > HL && zn < HL + ND;
    if (inDepth && b.y < GH + BR && ((Math.abs(px) < GW / 2) !== (Math.abs(b.x) <
    if (Math.abs(b.x) < GW / 2 && b.y < GH && zp <= HL + ND - BR && zn > HL + ND
    if (Math.abs(b.x) < GW / 2 && inDepth && (py - GH) * (b.y - GH) < 0) { b.y =
  }
  // استلام الكورة
```

```
  if (b.owner < 0 && S.phase === 'play') {
    let best = null, bd = 1e9;
    for (const p of S.players) {
      if (p.st === 1 || p.st === 2 || p.noTouch > 0) continue;
      const d = Math.hypot(b.x - p.x, b.z - p.z);
      const gkz = p.gk && inOwnBox(p);
      if (d < (gkz ? 1.45 : 1.05) && b.y < (gkz ? 2.7 : 1.05) && d < bd) { best =
    }
    if (best) {
      const spd = Math.hypot(b.vx, b.vy, b.vz);
      const lastP = b.last >= 0 ? S.players[b.last] : null;
      const fromOpp = lastP && lastP.team !== best.team;
      if (!best.gk && spd > 24 && fromOpp) {
        b.vx *= -0.3; b.vz *= -0.3; b.vy = Math.abs(b.vy) * 0.3 + 1.5; best.noTou
      } else if (best.gk && spd > 24 && fromOpp && Math.random() < 0.45) {
        b.vx = -b.vx * 0.3 + rnd(-7, 7); b.vz = -b.vz * 0.35; b.vy = 4.5; best.no
      } else {
        b.owner = best.id; b.last = best.id; best.protect = 0.7; best.holdT = 0;
        if (best.bot) best.think = rnd(0.15, 0.35);
        S.ev.push(['k', 0.15]);
      }
    }
  }
}

function scoreGoal(team) {
  const b = S.ball;
  S.score[team]++;
  const sc = b.last >= 0 ? S.players[b.last] : null;
  const txt = sc ? (sc.team === team ? sc.name : sc.name + ' (هدف عكسي)') : '';
  S.phase = 'goal'; S.phaseT = 3.4; S.kickTeam = 1 - team; S.goalTeam = team;
  b.owner = -1;
  S.ev.push(['g', team, txt]);
}

/* ---------- أفعال اللاعب ---------- */
function kickBall(p, dx, dz, speed, vy) {
  const b = S.ball, l = Math.hypot(dx, dz) || 1;
  dx /= l; dz /= l;
  b.owner = -1;
  b.x = p.x + dx * 0.9; b.z = p.z + dz * 0.9; b.y = Math.max(b.y, BR);
  b.vx = dx * speed; b.vz = dz * speed; b.vy = vy; b.last = p.id;
  p.noTouch = 0.35; p.rot = Math.atan2(dx, dz); p.holdT = 0;
  S.ev.push(['k', speed / 34]);
}
function aimDir(p) {
```

```
  const m = Math.hypot(p.input.mx, p.input.mz);
  if (!p.bot && m > 0.3) return [p.input.mx / m, p.input.mz / m];
  return [Math.sin(p.rot), Math.cos(p.rot)];
}
function actShoot(p, pow) {
  const b = S.ball;
  const own = b.owner === p.id;
  const close = b.owner < 0 && Math.hypot(b.x - p.x, b.z - p.z) < 1.7 && b.y < 1.
  if (!own && !close) return;
  let [dx, dz] = aimDir(p);
  const s = sgn(p.team), gx = -p.x, gz = HL * s - p.z, gl = Math.hypot(gx, gz);
  if ((dx * gx + dz * gz) / gl > 0.5 && gl < 40) { dx = dx * 0.6 + (gx / gl) * 0.
  kickBall(p, dx, dz, 15 + 20 * pow, 1.4 + 5.2 * pow);
}
function pickPassTarget(p) {
  const [ax, az] = aimDir(p);
  let best = null, bs = -1e9;
  for (const t of S.players) {
    if (t === p || t.team !== p.team || t.st === 2) continue;
    const vx = t.x - p.x, vz = t.z - p.z, d = Math.hypot(vx, vz) || 1;
    const dot = (vx * ax + vz * az) / d;
    const calling = t.call > 0;
    if (dot < 0.3 && !(calling && dot > -0.3)) continue;
    const sc = dot * 2 - d * 0.02 + (calling ? 1.6 : 0) - (t.gk ? 1.5 : 0);
    if (sc > bs) { bs = sc; best = t; }
  }
  return best;
}
function doPass(p, t) {
  const lx = t.x + t.vx * 0.45, lz = t.z + t.vz * 0.45;
  const dx = lx - p.x, dz = lz - p.z, d = Math.hypot(dx, dz);
  const lob = d > 24;
  kickBall(p, dx, dz, clamp(d * (lob ? 0.62 : 0.95) + 4, 9, 28), lob ? 6.5 : 0.6)
}
function actPass(p) {
  if (S.ball.owner !== p.id) return;
  const t = pickPassTarget(p);
  if (t) doPass(p, t);
  else { const [dx, dz] = aimDir(p); kickBall(p, dx, dz, 16, 0.6); }
}
function startSlide(p, dir) {
  p.st = 1; p.stT = 0.6; p.slideCd = 1.7; p.rot = dir;
  p.vx = Math.sin(dir) * 17; p.vz = Math.cos(dir) * 17;
  S.ev.push(['s']);
}
function startDash(p) {
```

```
  p.st = 3; p.stT = 0.4; p.slideCd = 1.5; p.protect = 0.8;
  S.ev.push(['s']);
}
function actSlide(p) {
  if (p.st !== 0 || p.slideCd > 0) return;
  if (S.ball.owner === p.id) {
    const m = Math.hypot(p.input.mx, p.input.mz);
    if (m > 0.2) p.rot = Math.atan2(p.input.mx, p.input.mz);
    startDash(p);
    return;
  }
  const [dx, dz] = aimDir(p);
  startSlide(p, Math.atan2(dx, dz));
}
function actCall(p) {
  p.call = 2.4;
  S.ev.push(['c', p.id]);
  const b = S.ball;
  if (b.owner >= 0) {
    const o = S.players[b.owner];
    if (o.bot && o.team === p.team && o !== p) { o.passTo = p.id; o.passDelay = 0
  }
}
function slideHits(p) {
  if (p.stT < 0.08) return;
  const b = S.ball;
  for (const q of S.players) {
    if (q === p || q.st === 2 || q.team === p.team) continue;
    if (q.gk && b.owner === q.id && inOwnBox(q)) continue;
    if (Math.hypot(q.x - p.x, q.z - p.z) < 1.15) {
      q.st = 2; q.stT = 1.4; q.vx = p.vx * 0.4; q.vz = p.vz * 0.4; q.charge = 0;
      S.ev.push(['t', q.id]);
      if (b.owner === q.id) {
        b.owner = -1; q.noTouch = 1.0;
        b.vx = Math.sin(p.rot) * 9 + rnd(-2, 2); b.vz = Math.cos(p.rot) * 9 + rnd
      }
    }
  }
  if (b.owner < 0 && p.noTouch <= 0 && b.y < 0.9 && Math.hypot(b.x - p.x, b.z - p
    b.vx = Math.sin(p.rot) * 12 + p.vx * 0.2; b.vz = Math.cos(p.rot) * 12 + p.vz
    p.noTouch = 0.3; S.ev.push(['k', 0.4]);
  }
}

/* ================= ذكاء البوتات ================= */
function moveTo(p, tx, tz, maxM, allowSprint) {
```

```
  const inp = p.input, dx = tx - p.x, dz = tz - p.z, d = Math.hypot(dx, dz);
  if (d < 0.5) {
    inp.mx = inp.mz = 0; inp.sp = 0;
    const b = S.ball; p.rot = lerpAng(p.rot, Math.atan2(b.x - p.x, b.z - p.z), 0.
    return;
  }
  const m = Math.min(maxM, d / 2.5);
  inp.mx = (dx / d) * m; inp.mz = (dz / d) * m;
  inp.sp = allowSprint && d > 4 && p.stam > 0.3 ? 1 : 0;
}
function chaseRank(p) {
  const b = S.ball, px = b.x + b.vx * 0.2, pz = b.z + b.vz * 0.2;
  const my = Math.hypot(px - p.x, pz - p.z);
  let r = 0;
  for (const q of S.players) {
    if (q === p || q.team !== p.team || q.gk || q.st === 2) continue;
    if (Math.hypot(px - q.x, pz - q.z) < my) r++;
  }
  return r;
}
function nearestOpp(p) {
  let n = null, nd = 1e9;
  for (const q of S.players) {
    if (q.team === p.team || q.st === 2) continue;
    const d = dist(p, q); if (d < nd) { nd = d; n = q; }
  }
  return [n, nd];
}
function bestPassTarget(p) {
  const s = sgn(p.team);
  let best = null, bs = -1e9;
  for (const t of S.players) {
    if (t === p || t.team !== p.team || t.gk || t.st === 2) continue;
    const d = dist(p, t); if (d < 4) continue;
    let open = 8;
    for (const q of S.players) if (q.team !== p.team) open = Math.min(open, dist(
    let sc = (t.z - p.z) * s * 0.15 + open * 0.3 - d * 0.04 + (t.call > 0 ? 4 : 0
    // هل في خصم بالطريق؟
    const dx = t.x - p.x, dz = t.z - p.z;
    for (const q of S.players) {
      if (q.team === p.team) continue;
      const u = clamp(((q.x - p.x) * dx + (q.z - p.z) * dz) / (d * d), 0, 1);
      const cx = p.x + dx * u, cz = p.z + dz * u;
      if (Math.hypot(q.x - cx, q.z - cz) < 1.3 && u > 0.1 && u < 0.95) sc -= 2.5;
    }
    if (sc > bs) { bs = sc; best = t; }
```

```
  }
  return best;
}
function botThink(p, dt) {
  const b = S.ball, s = sgn(p.team), inp = p.input;
  inp.sp = 0;
  p.think -= dt; const decide = p.think <= 0; if (decide) p.think = rnd(0.2, 0.4)
  if (p.st === 1 || p.st === 2) { inp.mx = inp.mz = 0; return; }
  if (S.phase !== 'play') { moveTo(p, p.x, p.z, 0, false); return; }
  const own = b.owner === p.id;
  const ownerP = b.owner >= 0 ? S.players[b.owner] : null;
  const teamBall = ownerP && ownerP.team === p.team;
  const oppBall = ownerP && ownerP.team !== p.team;
  if (p.gk) return gkThink(p, dt, own, oppBall);
  if (own) return ownerThink(p, dt, decide);
  const rank = chaseRank(p);
  if (!teamBall && (rank === 0 || (oppBall && rank === 1))) {
    let tx = b.x + b.vx * 0.25, tz = b.z + b.vz * 0.25;
    if (oppBall && rank === 1) { const gz = -HL * s; tx = b.x * 0.7; tz = b.z + (
    moveTo(p, tx, tz, 1, true);
    if (oppBall && rank === 0 && p.slideCd <= 0 && decide && p.st === 0) {
      const d = dist(p, ownerP);
      if (d < 3.0 && d > 0.9 && Math.random() < 0.18) {
        startSlide(p, Math.atan2(ownerP.x + ownerP.vx * 0.15 - p.x, ownerP.z + ow
      }
    }
    return;
  }
  const f = FORM[p.slot];
  let tx = f[0] * 0.75 + b.x * 0.35;
  let tz = f[1] * s + b.z * 0.5 + (teamBall ? 7 * s : 0) + (teamBall && p.slot >=
  tx = clamp(tx, -HW + 2, HW - 2); tz = clamp(tz, -HL + 4, HL - 4);
  moveTo(p, tx, tz, 0.85, false);
}
function ownerThink(p, dt, decide) {
  const s = sgn(p.team), gz = HL * s, inp = p.input;
  const dxg = -p.x, dzg = gz - p.z, dg = Math.hypot(dxg, dzg);
  p.holdT += dt;
  if (p.passTo >= 0) {
    p.passDelay -= dt;
    if (p.passDelay <= 0) {
      const t = S.players[p.passTo]; p.passTo = -1;
      if (t && t.team === p.team && t.st !== 2) { doPass(p, t); return; }
    }
  }
  const [near, nd] = nearestOpp(p);
```

```
  if (decide) {
    if (dg < 24 && Math.random() < (dg < 13 ? 0.9 : 0.4)) {
      let gkx = 0;
      for (const q of S.players) if (q.team !== p.team && q.gk) gkx = q.x;
      const aimX = (gkx > 0 ? -1 : 1) * (GW / 2 - 1.1) * rnd(0.55, 1);
      kickBall(p, aimX - p.x, gz - p.z, rnd(26, 34), rnd(1.5, 4.8));
      return;
    }
    const pressured = near && nd < 2.6;
    if (pressured || p.holdT > rnd(2.5, 5)) {
      const t = bestPassTarget(p);
      if (t && (!pressured || Math.random() < 0.6)) { doPass(p, t); return; }
      if (pressured && p.slideCd <= 0 && p.st === 0 && Math.random() < 0.5) start
    }
  }
  let mx = dxg / dg, mz = dzg / dg;
  if (near && nd < 4) {
    const ax = p.x - near.x, az = p.z - near.z, al = Math.hypot(ax, az) || 1, w =
    mx += (ax / al) * w; mz += (az / al) * w * 0.4;
  }
  if (Math.abs(p.x) > HW - 3) mx -= Math.sign(p.x) * 0.7;
  const l = Math.hypot(mx, mz) || 1;
  inp.mx = mx / l; inp.mz = mz / l; inp.sp = p.stam > 0.35 ? 1 : 0;
}
function gkThink(p, dt, own, oppBall) {
  const s = sgn(p.team), gz = -HL * s, b = S.ball, inp = p.input;
  if (own) {
    p.holdT += dt; inp.mx = inp.mz = 0;
    p.rot = lerpAng(p.rot, s > 0 ? 0 : Math.PI, 0.2);
    if (p.holdT > 1.1) {
      let t = null;
      if (p.passTo >= 0) { t = S.players[p.passTo]; p.passTo = -1; }
      if (!t) t = bestPassTarget(p);
      if (t) doPass(p, t); else kickBall(p, rnd(-0.3, 0.3), s, 24, 6);
    }
    return;
  }
  const bz = b.z - gz, free = b.owner < 0;
  const inBox = Math.abs(b.x) < 11 && Math.abs(bz) < 10;
  if (inBox && free && Math.hypot(b.x - p.x, b.z - p.z) < 7) { moveTo(p, b.x, b.z
  if (inBox && oppBall && Math.hypot(b.x - p.x, b.z - p.z) < 4) { moveTo(p, b.x,
  if (free && b.vz * s < -8) {
    const t = (gz + 0.9 * s - b.z) / b.vz;
    if (t > 0 && t < 1.6) {
      moveTo(p, clamp(b.x + b.vx * t, -GW / 2, GW / 2), gz + 0.9 * s, 1, true); i
    }
```

```
  }
  const tx = clamp(b.x * 0.35, -GW / 2 + 0.7, GW / 2 - 0.7);
  const tz = gz + s * (1.2 + clamp(Math.abs(bz) / 25, 0, 1) * 1.6);
  moveTo(p, tx, tz, 1, false);
}

/* ================== الشبكة (أونلاين) ================== */
const NET = { mode: 'off', peer: null, conns: [], hostConn: null, room: '', pub:

function hasPeer() { return typeof window.Peer === 'function'; }
function makePeer(id) {
  return new Promise((res, rej) => {
    let done = false;
    const peer = id ? new Peer(id, { debug: 0 }) : new Peer({ debug: 0 });
    const t = setTimeout(() => { if (!done) { done = true; try { peer.destroy();
    peer.on('open', () => { if (done) return; done = true; clearTimeout(t); res(p
    peer.on('error', (e) => { if (done) return; done = true; clearTimeout(t); try
  });
}
function connectTo(peer, room, timeout) {
  return new Promise((res, rej) => {
    let done = false;
    const conn = peer.connect(PFX + room, { reliable: true, serialization: 'json'
    conn._q = [];
    const fail = (e) => {
      if (done) return; done = true; clearTimeout(t); peer.off('error', onErr);
      try { conn.close(); } catch (x) {}
      rej(e);
    };
    const onErr = (e) => { if (e && e.type === 'peer-unavailable') fail(e); };
    peer.on('error', onErr);
    const t = setTimeout(() => fail({ type: 'timeout' }), timeout || 7000);
    conn.on('open', () => { conn.send({ t: 'h', n: playerName() }); });
    conn.on('data', (d) => {
      if (!done) {
        if (d && d.t === 'w') { done = true; clearTimeout(t); peer.off('error', o
        else if (d && d.t === 'full') fail({ type: 'full' });
        return;
      }
      if (NET.mode === 'client' && NET.hostConn === conn) clientData(d); else con
    });
  });
}

function freeSlot() {
  const hum = [0, 0];
```

```
  for (const p of S.players) if (!p.bot) hum[p.team]++;
  const first = hum[0] <= hum[1] ? 0 : 1;
  for (const t of [first, 1 - first]) for (const k of HUMAN_ORDER) { const p = S.
  return null;
}
function claim(p, name) {
  p.bot = false; p.name = (name || 'لاعب').slice(0, 14); p.prev = null; p.prevSh =
  p.input = zeroInput(); p.passTo = -1; p.charge = 0; S.rosterVer++;
}
function releaseSlot(p) {
  p.bot = true; p.name = botName(p.id); p.conn = null; p.input = zeroInput(); p.p
}
function claimLocal() {
  const p = freeSlot(); claim(p, playerName());
  p.input = LI; myId = p.id;
}

function startHost(peer, room, pub) {
  NET.mode = 'host'; NET.peer = peer; NET.room = room; NET.pub = pub; NET.conns =
  S = newState(); S.autoRestart = pub; claimLocal(); placeKickoff();
  peer.on('connection', (conn) => {
    conn.on('data', (d) => hostData(conn, d));
    conn.on('close', () => hostDrop(conn));
    conn.on('error', () => hostDrop(conn));
  });
  peer.on('disconnected', () => { try { if (!peer.destroyed) peer.reconnect(); }
  peer.on('error', () => {});
  startGame();
}
function hostData(conn, d) {
  if (!d || NET.mode !== 'host') return;
  if (d.t === 'h') {
    if (conn._pid != null) return;
    const p = freeSlot();
    if (!p) { conn.send({ t: 'full' }); setTimeout(() => { try { conn.close(); }
    claim(p, String(d.n || 'لاعب'));
    p.conn = conn; conn._pid = p.id; NET.conns.push(conn);
    conn.send({ t: 'w', id: p.id, room: NET.room, pub: NET.pub });
    sendRoster(true);
    toast(p.name + ' دخل المباراة');
  } else if (d.t === 'i') {
    const p = conn._pid != null ? S.players[conn._pid] : null;
    if (!p || p.conn !== conn) return;
    const i = p.input;
    i.mx = clamp(+d.mx || 0, -1, 1); i.mz = clamp(+d.mz || 0, -1, 1);
    i.sp = d.sp ? 1 : 0; i.sh = d.sh ? 1 : 0; i.pa = d.pa | 0; i.sl = d.sl | 0; i
```

```
  } else if (d.t === 'pg') {
    try { conn.send({ t: 'po', ts: d.ts }); } catch (e) {}
  }
}
function hostDrop(conn) {
  const i = NET.conns.indexOf(conn);
  if (i >= 0) NET.conns.splice(i, 1);
  if (conn._pid != null && S) {
    const p = S.players[conn._pid];
    if (p && p.conn === conn) { toast(p.name + ' طلع من المباراة'); releaseSlot(p
    conn._pid = null;
  }
}
function sendRoster(force) {
  if (!force && NET.rosterSent === S.rosterVer) return;
  NET.rosterSent = S.rosterVer;
  const msg = { t: 'r', l: S.players.map((p) => [p.name, p.bot ? 0 : 1]) };
  for (const c of NET.conns) { try { c.send(msg); } catch (e) {} }
}
function snapshot() {
  const P = [];
  for (const p of S.players) P.push(r2(p.x), r2(p.z), r2(p.rot), p.st, p.call > 0
  const b = S.ball;
  return { t: 's', p: P, b: [r2(b.x), r2(b.y), r2(b.z), r2(b.vx), r2(b.vy), r2(b.
}
function broadcast() {
  sendRoster(false);
  if (!NET.conns.length) { NET.netEv = []; return; }
  const msg = snapshot();
  for (const c of NET.conns) { try { c.send(msg); } catch (e) {} }
  NET.netEv = [];
}

function startClient(peer, conn, w, room) {
  NET.mode = 'client'; NET.peer = peer; NET.hostConn = conn; NET.room = room; NET
  S = newState(); myId = w.id; S.players[myId].bot = false; S.players[myId].name
  for (const p of S.players) { p.tx = p.x; p.tz = p.z; p.trot = p.rot; }
  conn.on('close', () => { if (NET.mode === 'client' && NET.hostConn === conn) ba
  peer.on('disconnected', () => { try { if (!peer.destroyed) peer.reconnect(); }
  peer.on('error', () => {});
  startGame();
  const q = conn._q; conn._q = [];
  for (const d of q) clientData(d);
}
function clientData(d) {
  if (!d || !S) return;
```

```
  if (d.t === 's') {
    const P = d.p;
    for (let i = 0; i < NP; i++) {
      const p = S.players[i], o = i * 6;
      p.tx = P[o]; p.tz = P[o + 1]; p.trot = P[o + 2]; p.st = P[o + 3]; p.call =
      p.x = p.tx; p.z = p.tz;
    }
    const b = S.ball, B = d.b;
    b.x = B[0]; b.y = B[1]; b.z = B[2]; b.vx = B[3]; b.vy = B[4]; b.vz = B[5]; b.
    S.score = d.sc; S.time = d.tm; S.phase = d.ph; S.goalTeam = d.gt;
    if (d.ev) for (const e of d.ev) handleEvent(e);
  } else if (d.t === 'r') {
    d.l.forEach((r, i) => { const p = S.players[i]; p.name = r[0]; p.bot = !r[1];
    S.rosterVer++;
  } else if (d.t === 'po') {
    NET.ping = Math.round(performance.now() - d.ts);
  }
}
function sendInput() {
  const c = NET.hostConn; if (!c || !c.open) return;
  try { c.send({ t: 'i', mx: r2(LI.mx), mz: r2(LI.mz), sp: LI.sp, sh: LI.sh, pa:
}
function netClose() {
  const peer = NET.peer;
  NET.mode = 'off'; NET.peer = null; NET.conns = []; NET.hostConn = null; NET.roo
  if (peer) { try { peer.destroy(); } catch (e) {} }
}

let busy = false;
async function quickPlay() {
  if (busy) return;
  if (!hasPeer()) { status('ما قدرنا نحمّل نظام الأونلاين، تأكد من النت وحدّث الصفحة
  busy = true; status('جاري البحث عن مباراة…');
  try {
    let cp = null;
    try { cp = await makePeer(null); } catch (e) { status('السيرفر، جرّب مرة ثانية
    for (const r of PUB_ROOMS) {
      try { const { conn, welcome } = await connectTo(cp, r, 6000); status(''); s
    }
    try { cp.destroy(); } catch (e) {}
    status('ما لقينا أحد، بنسوي مباراة وننتظر لاعبين…');
    for (const r of PUB_ROOMS) {
      try { const hp = await makePeer(PFX + r); status(''); startHost(hp, r, true
      catch (e) { if (e.type !== 'unavailable-id') break; }
    }
    status('صار خطأ بالاتصال، جرّب مرة ثانية');
```

```
  } finally { busy = false; }
}
async function hostRoom() {
  if (busy) return;
  if (!hasPeer()) { status('ما قدرنا نحمّل نظام الأونلاين، تأكد من النت وحدّث الصفحة
  busy = true; status('جاري إنشاء الغرفة…');
  try {
    for (let i = 0; i < 4; i++) {
      const code = String(Math.floor(1000 + Math.random() * 9000));
      try { const hp = await makePeer(PFX + code); status(''); startHost(hp, code
      catch (e) { if (e.type !== 'unavailable-id') break; }
    }
    status('ما قدرنا نسوي غرفة، جرّب مرة ثانية');
  } finally { busy = false; }
}
async function joinRoom(code) {
  if (busy) return;
  code = String(code || '').replace(/[^\d٩-٠]/g, '').replace(/[٩-٠]/g, (c) => '٠١
  if (code.length !== 4) { status('اكتب رمز الغرفة (٤ أرقام)'); return; }
  if (!hasPeer()) { status('ما قدرنا نحمّل نظام الأونلاين، تأكد من النت وحدّث الصفحة
  busy = true; status('جاري الدخول للغرفة ' + code + '…');
  try {
    let cp;
    try { cp = await makePeer(null); } catch (e) { status('السيرفر، جرّب مرة ثانية
    try { const { conn, welcome } = await connectTo(cp, code, 8000); status('');
    catch (e) {
      try { cp.destroy(); } catch (x) {}
      status(e.type === 'full' ? 'الغرفة مليانة' : 'ما لقينا الغرفة، تأكد من الرمز
    }
  } finally { busy = false; }
}

/* ================== الرسم ثري دي ================== */
const canvas = $('c');
const renderer = new THREE.WebGLRenderer({ canvas, antialias: true, powerPreferen
renderer.setPixelRatio(Math.min(window.devicePixelRatio || 1, 1.5));
renderer.shadowMap.enabled = true;
renderer.shadowMap.type = THREE.PCFShadowMap;
const scene = new THREE.Scene();
scene.fog = new THREE.Fog(0xf7c27e, 70, 300);
const camera = new THREE.PerspectiveCamera(58, 1, 0.1, 900);
const cam = { x: 0, y: 30, z: -40, lx: 0, ly: 0, lz: 0, orbit: 0, snap: true };

function canvasTex(w, h, draw) {
  const c = document.createElement('canvas'); c.width = w; c.height = h;
  draw(c.getContext('2d'), w, h);
```

```
  const t = new THREE.CanvasTexture(c);
  t.anisotropy = Math.min(8, renderer.capabilities.getMaxAnisotropy());
  return t;
}

const MESH = [], R = [];
let ballMesh, ballShadow;
const tmpV = new THREE.Vector3();

function buildWorld() {
  // السماء
  const sky = new THREE.Mesh(new THREE.SphereGeometry(600, 32, 16), new THREE.Sha
    side: THREE.BackSide, depthWrite: false, fog: false,
    uniforms: { top: { value: new THREE.Color(0xffd98a) }, mid: { value: new THRE
    vertexShader: 'varying vec3 vP; void main(){ vP = position; gl_Position = pro
    fragmentShader: 'uniform vec3 top; uniform vec3 mid; uniform vec3 bot; varyin
  }));
  scene.add(sky);
  // الشمس
  const sun = new THREE.Sprite(new THREE.SpriteMaterial({ map: canvasTex(128, 128
    const gr = g.createRadialGradient(64, 64, 4, 64, 64, 64);
    gr.addColorStop(0, 'rgba(255,255,240,1)'); gr.addColorStop(0.25, 'rgba(255,24
    g.fillStyle = gr; g.fillRect(0, 0, w, w);
  }), fog: false, depthWrite: false, transparent: true }));
  sun.position.set(-160, 70, 380); sun.scale.set(120, 120, 1); scene.add(sun);

  // الإضاءة
  scene.add(new THREE.HemisphereLight(0xfff0d0, 0xb8703a, 0.78));
  const dl = new THREE.DirectionalLight(0xfff1d6, 0.8);
  dl.position.set(-30, 55, 25); dl.castShadow = true;
  dl.shadow.mapSize.set(1024, 1024);
  Object.assign(dl.shadow.camera, { left: -45, right: 45, top: 50, bottom: -50, n
  dl.shadow.camera.updateProjectionMatrix();
  scene.add(dl);

  // الأرض
  const ground = new THREE.Mesh(new THREE.PlaneGeometry(900, 900), new THREE.Mesh
  ground.rotation.x = -Math.PI / 2; ground.position.y = -0.03; ground.receiveShad

  // الملعب
  const PX = 768 / FW;
  const fieldTex = canvasTex(768, Math.round(FL * PX), (g, w, h) => {
    const bands = 14;
    for (let i = 0; i < bands; i++) { g.fillStyle = i % 2 ? '#dd8a40' : '#e6974c'
    g.strokeStyle = 'rgba(255,255,255,.9)'; g.lineWidth = 4; g.fillStyle = 'rgba(
    const X = (x) => (x + HW) * PX, Z = (z) => (z + HL) * PX;
```

```
  g.strokeRect(X(-HW) + 2, Z(-HL) + 2, w - 4, h - 4);
  g.beginPath(); g.moveTo(0, Z(0)); g.lineTo(w, Z(0)); g.stroke();
  g.beginPath(); g.arc(X(0), Z(0), 6 * PX, 0, Math.PI * 2); g.stroke();
  g.beginPath(); g.arc(X(0), Z(0), 0.35 * PX, 0, Math.PI * 2); g.fill();
  for (const sd of [-1, 1]) {
    const gz = sd * HL;
    g.strokeRect(X(-10), sd < 0 ? Z(gz) : Z(gz - 9), 20 * PX, 9 * PX);
    g.strokeRect(X(-5.5), sd < 0 ? Z(gz) : Z(gz - 3.5), 11 * PX, 3.5 * PX);
    g.beginPath(); g.arc(X(0), Z(gz - sd * 7), 0.3 * PX, 0, Math.PI * 2); g.fil
    g.beginPath(); g.arc(X(0), Z(gz - sd * 7), 5 * PX, sd < 0 ? 0.2 * Math.PI :
  }
});
const field = new THREE.Mesh(new THREE.PlaneGeometry(FW, FL), new THREE.MeshLam
field.rotation.x = -Math.PI / 2; field.position.y = 0.0; field.receiveShadow =
const apron = new THREE.Mesh(new THREE.PlaneGeometry(BX * 2 + 1, BZ * 2 + 1), n
apron.rotation.x = -Math.PI / 2; apron.position.y = -0.015; apron.receiveShadow

// الحواجز الإعلانية
const adTex = canvasTex(1024, 64, (g, w, h) => {
  g.fillStyle = '#2a1a10'; g.fillRect(0, 0, w, h);
  g.fillStyle = '#ffc23d'; g.fillRect(0, 0, w, 6); g.fillRect(0, h - 6, w, 6);
  g.font = '900 34px Cairo, sans-serif'; g.textAlign = 'center'; g.textBaseline
  g.fillText('⚽ كورة الصحراء ⚽ أونلاين ⚽ كورة الصحراء ⚽', w / 2, h / 2
});
adTex.wrapS = THREE.RepeatWrapping;
const boardH = 1.1;
const mkBoard = (len, x, z, ry) => {
  const t = adTex.clone(); t.needsUpdate = true; t.wrapS = THREE.RepeatWrapping
  const m = new THREE.Mesh(new THREE.BoxGeometry(len, boardH, 0.35), [0, 0, 0,
  m.position.set(x, boardH / 2, z); m.rotation.y = ry; m.castShadow = true; m.r
};
mkBoard(BZ * 2 + 0.7, BX + 0.18, 0, Math.PI / 2);
mkBoard(BZ * 2 + 0.7, -BX - 0.18, 0, Math.PI / 2);
mkBoard(BX * 2 + 0.7, 0, BZ + 0.18, 0);
mkBoard(BX * 2 + 0.7, 0, -BZ - 0.18, 0);

// المرمى
const netTex = canvasTex(64, 64, (g, w) => { g.clearRect(0, 0, w, w); g.strokeS
netTex.wrapS = netTex.wrapT = THREE.RepeatWrapping; netTex.repeat.set(6, 4);
const netM = new THREE.MeshBasicMaterial({ map: netTex, transparent: true, side
const postM = new THREE.MeshLambertMaterial({ color: 0xffffff });
for (const sd of [-1, 1]) {
  const gz = sd * HL, grp = new THREE.Group();
  const postG = new THREE.CylinderGeometry(0.13, 0.13, GH, 10);
  for (const px of [-GW / 2, GW / 2]) {
    const pm = new THREE.Mesh(postG, postM); pm.position.set(px, GH / 2, gz); p
```

```
    const bp = new THREE.Mesh(new THREE.CylinderGeometry(0.05, 0.05, GH, 6), po
  }
  const bar = new THREE.Mesh(new THREE.CylinderGeometry(0.13, 0.13, GW + 0.26,
  bar.rotation.z = Math.PI / 2; bar.position.set(0, GH, gz); bar.castShadow = t
  const back = new THREE.Mesh(new THREE.PlaneGeometry(GW, GH), netM); back.posi
  const top = new THREE.Mesh(new THREE.PlaneGeometry(GW, ND), netM); top.rotati
  for (const px of [-GW / 2, GW / 2]) {
    const sideN = new THREE.Mesh(new THREE.PlaneGeometry(ND, GH), netM); sideN.
  }
  scene.add(grp);
}
// رايات الزوايا
const flagM = new THREE.MeshLambertMaterial({ color: 0xff3b30, side: THREE.Doub
for (const fx of [-HW, HW]) for (const fz of [-HL, HL]) {
  const pole = new THREE.Mesh(new THREE.CylinderGeometry(0.04, 0.04, 1.6, 6), p
  const fl = new THREE.Mesh(new THREE.PlaneGeometry(0.5, 0.35), flagM); fl.posi
}

// الطبيعة: جبال وصخور وصبار
const rockMs = [0xd8733a, 0xc8642f, 0xe08548, 0xb9582a].map((c) => new THREE.Me
let seed = 7; const srnd = () => { seed = (seed * 16807) % 2147483647; return s
for (let i = 0; i < 22; i++) {
  const a = srnd() * Math.PI * 2, d = 95 + srnd() * 110, h = 10 + srnd() * 26,
  const m = new THREE.Mesh(new THREE.CylinderGeometry(r * (0.55 + srnd() * 0.3)
  m.position.set(Math.cos(a) * d, h / 2 - 0.5, Math.sin(a) * d); m.rotation.y =
}
for (let i = 0; i < 14; i++) {
  const a = srnd() * Math.PI * 2, d = 260 + srnd() * 80, h = 40 + srnd() * 60;
  const m = new THREE.Mesh(new THREE.ConeGeometry(50 + srnd() * 40, h, 5), rock
  m.position.set(Math.cos(a) * d, h / 2 - 2, Math.sin(a) * d); m.rotation.y = s
}
const cactM = new THREE.MeshLambertMaterial({ color: 0x4f9a3a, flatShading: tru
for (let i = 0; i < 30; i++) {
  const a = srnd() * Math.PI * 2, d = 38 + srnd() * 50;
  const x = Math.cos(a) * d, z = Math.sin(a) * d;
  if (Math.abs(x) < BX + 4 && Math.abs(z) < BZ + 4) continue;
  const g = new THREE.Group(), h = 2 + srnd() * 2.5;
  const trunk = new THREE.Mesh(new THREE.CylinderGeometry(0.35, 0.4, h, 7), cac
  for (const sd of [-1, 1]) if (srnd() > 0.35) {
    const ah = 0.8 + srnd(), arm = new THREE.Mesh(new THREE.CylinderGeometry(0.
    arm.position.set(sd * 0.7, h * 0.55 + ah / 2, 0); g.add(arm);
    const j = new THREE.Mesh(new THREE.CylinderGeometry(0.2, 0.2, 0.6, 6), cact
  }
  g.position.set(x, 0, z); g.rotation.y = srnd() * 6; g.traverse((o) => { o.cas
}
```

```
  // الكورة
  const ballTex = canvasTex(256, 128, (g, w, h) => {
    g.fillStyle = '#ffffff'; g.fillRect(0, 0, w, h);
    g.fillStyle = '#151515';
    const spots = [[0.1, 0.5], [0.35, 0.5], [0.6, 0.5], [0.85, 0.5], [0.22, 0.15]
    for (const [u, v] of spots) {
      g.beginPath();
      for (let k = 0; k < 5; k++) { const an = (k / 5) * Math.PI * 2 - Math.PI /
      g.closePath(); g.fill();
    }
    g.fillRect(0, 0, w, 4); g.fillRect(0, h - 4, w, 4);
  });
  ballMesh = new THREE.Mesh(new THREE.SphereGeometry(BR, 20, 14), new THREE.MeshL
  ballMesh.castShadow = true; scene.add(ballMesh);
  ballShadow = new THREE.Mesh(new THREE.CircleGeometry(BR * 1.1, 16), new THREE.M
  ballShadow.rotation.x = -Math.PI / 2; scene.add(ballShadow);

  // اللاعبين
  for (let i = 0; i < NP; i++) { MESH.push(makeChar(i)); R.push({ x: 0, z: 0, rot
}

const GEO = {};
function geo() {
  if (GEO.torso) return GEO;
  GEO.torso = new THREE.BoxGeometry(0.82, 0.78, 0.44);
  GEO.hips = new THREE.BoxGeometry(0.8, 0.22, 0.42);
  GEO.leg = new THREE.BoxGeometry(0.36, 0.66, 0.38); GEO.leg.translate(0, -0.33,
  GEO.shoe = new THREE.BoxGeometry(0.38, 0.16, 0.48); GEO.shoe.translate(0, -0.74
  GEO.sleeve = new THREE.BoxGeometry(0.3, 0.32, 0.32); GEO.sleeve.translate(0, -0
  GEO.arm = new THREE.BoxGeometry(0.26, 0.48, 0.28); GEO.arm.translate(0, -0.5, 0
  GEO.head = new THREE.BoxGeometry(0.56, 0.56, 0.56);
  GEO.hair = new THREE.BoxGeometry(0.6, 0.2, 0.6);
  GEO.hairBack = new THREE.BoxGeometry(0.6, 0.36, 0.14);
  GEO.eye = new THREE.BoxGeometry(0.08, 0.1, 0.02);
  GEO.ring = new THREE.RingGeometry(0.62, 0.82, 28); GEO.ring.rotateX(-Math.PI /
  return GEO;
}
function numTex(num, col, big) {
  const css = '#' + col.toString(16).padStart(6, '0');
  return canvasTex(128, 128, (g, w) => {
    g.fillStyle = css; g.fillRect(0, 0, w, w);
    g.fillStyle = 'rgba(255,255,255,.18)'; g.fillRect(0, 0, w, 10);
    g.fillStyle = '#fff'; g.strokeStyle = 'rgba(0,0,0,.35)'; g.lineWidth = 5;
    g.font = (big ? '900 78px' : '900 34px') + ' Cairo, Arial, sans-serif'; g.tex
    const y = big ? 70 : 44, x = big ? 64 : 88;
    g.strokeText(String(num), x, y); g.fillText(String(num), x, y);
```

```
  });
}
function labelSprite() {
  const c = document.createElement('canvas'); c.width = 256; c.height = 64;
  const t = new THREE.CanvasTexture(c);
  const sp = new THREE.Sprite(new THREE.SpriteMaterial({ map: t, transparent: tru
  sp.scale.set(2.8, 0.7, 1); sp.userData = { c, t, txt: null };
  return sp;
}
function setLabel(sp, txt, color) {
  const key = txt + color;
  if (sp.userData.txt === key) return;
  sp.userData.txt = key;
  const c = sp.userData.c, g = c.getContext('2d');
  g.clearRect(0, 0, c.width, c.height);
  g.font = '800 30px Cairo, sans-serif'; g.textAlign = 'center'; g.textBaseline =
  g.lineWidth = 6; g.strokeStyle = 'rgba(0,0,0,.7)'; g.strokeText(txt, 128, 34);
  g.fillStyle = color; g.fillText(txt, 128, 34);
  sp.userData.t.needsUpdate = true;
}
let bubbleTex = null;
function makeBubble() {
  if (!bubbleTex) bubbleTex = canvasTex(256, 112, (g) => {
    g.fillStyle = '#fff'; g.strokeStyle = '#1d5fd6'; g.lineWidth = 6;
    const r = 26, x = 8, y = 8, w = 240, h = 74;
    g.beginPath(); g.moveTo(x + r, y); g.arcTo(x + w, y, x + w, y + h, r); g.arcT
    g.lineTo(140, y + h); g.lineTo(128, y + h + 22); g.lineTo(114, y + h);
    g.arcTo(x, y + h, x, y, r); g.arcTo(x, y, x + w, y, r); g.closePath(); g.fill
    g.fillStyle = '#1d5fd6'; g.font = '900 44px Cairo, sans-serif'; g.textAlign =
    g.fillText('46 ,128 ,'!شوتولي);
  });
  const sp = new THREE.Sprite(new THREE.SpriteMaterial({ map: bubbleTex, transpar
  sp.scale.set(2.4, 1.05, 1); sp.renderOrder = 10;
  return sp;
}
function makeChar(i) {
  const G = geo(), p = S.players[i], team = p.team;
  const jersey = p.gk ? GK_COL[team] : TEAM_COL[team];
  const shortsC = team === 0 ? 0xf2f2f2 : 0x222222;
  const skin = SKINS[(i * 3) % SKINS.length], hair = HAIRS[(i * 5 + 1) % HAIRS.le
  const L = (c) => new THREE.MeshLambertMaterial({ color: c });
  const mJ = L(jersey), mS = L(shortsC), mSk = L(skin), mH = L(hair), mShoe = L(0
  const g = new THREE.Group(), body = new THREE.Group(); g.add(body);
  const torso = new THREE.Mesh(G.torso, [mJ, mJ, mJ, mJ, new THREE.MeshLambertMat
  torso.position.y = 1.27; torso.castShadow = true; body.add(torso);
  const hips = new THREE.Mesh(G.hips, mS); hips.position.y = 0.86; body.add(hips)
```

```
  const mkLeg = (x) => { const lg = new THREE.Group(); lg.position.set(x, 0.82, 0
  const mkArm = (x) => { const a = new THREE.Group(); a.position.set(x, 1.62, 0);
  const legL = mkLeg(-0.2), legR = mkLeg(0.2), armL = mkArm(-0.56), armR = mkArm(
  const head = new THREE.Mesh(G.head, mSk); head.position.y = 1.96; head.castShad
  const hr = new THREE.Mesh(G.hair, mH); hr.position.y = 2.3; body.add(hr);
  const hb = new THREE.Mesh(G.hairBack, mH); hb.position.set(0, 2.08, -0.25); bod
  for (const ex of [-0.12, 0.12]) { const e = new THREE.Mesh(G.eye, mEye); e.posi
  const ring = new THREE.Mesh(G.ring, new THREE.MeshBasicMaterial({ color: TEAM_C
  ring.position.y = 0.03; g.add(ring);
  const label = labelSprite(); label.position.y = 2.95; g.add(label);
  const bubble = makeBubble(); bubble.position.y = 3.75; bubble.visible = false;
  scene.add(g);
  return { g, body, legL, legR, armL, armR, ring, label, bubble };
}

let clockT = 0;
function animateChar(m, p, r, dt) {
  const st = p.st;
  let rx = 0, y = 0, leg = 0, arm = 0, up = false, fwdArms = false;
  if (st === 1) { rx = -1.22; y = 0.12; leg = -0.35; }
  else if (st === 2) { rx = 1.42; y = 0.22; }
  else {
    r.ph += dt * r.spd * 1.45;
    const amp = Math.min(1, r.spd / 7) * 0.95;
    leg = Math.sin(r.ph) * amp; arm = -leg * 0.8;
    if (st === 3) rx = 0.3;
    if (S.phase === 'goal' && p.team === S.goalTeam) { up = true; y = Math.abs(Ma
    if (p.gk && S.ball.owner === p.id) fwdArms = true;
  }
  const k = Math.min(1, dt * 14);
  m.body.rotation.x += (rx - m.body.rotation.x) * k;
  m.body.position.y += (y - m.body.position.y) * k;
  m.legL.rotation.x = leg; m.legR.rotation.x = -leg;
  if (up) { m.armL.rotation.x = -Math.PI + 0.25; m.armR.rotation.x = -Math.PI + 0
  else if (fwdArms) { m.armL.rotation.x = -1.45; m.armR.rotation.x = -1.45; m.arm
  else { m.armL.rotation.x = arm; m.armR.rotation.x = -arm; m.armL.rotation.z = -
}

const ballR = { x: 0, y: BR, z: 0 };
function render(dt) {
  clockT += dt;
  const client = NET.mode === 'client';
  const k = 1 - Math.exp(-14 * dt);
  for (let i = 0; i < NP; i++) {
    const p = S.players[i], r = R[i], m = MESH[i];
    if (client) {
```

```
      const dx = p.tx - r.x, dz = p.tz - r.z;
      if (dx * dx + dz * dz > 64) { r.x = p.tx; r.z = p.tz; r.lx = r.x; r.lz = r.
      r.rot = lerpAng(r.rot, p.trot, k);
    } else { r.x = p.x; r.z = p.z; r.rot = p.rot; }
    const sp = Math.hypot(r.x - r.lx, r.z - r.lz) / Math.max(dt, 1e-4);
    r.lx = r.x; r.lz = r.z;
    r.spd += (Math.min(sp, 20) - r.spd) * Math.min(1, dt * 10);
    m.g.position.set(r.x, 0, r.z); m.g.rotation.y = r.rot;
    animateChar(m, p, r, dt);
    const me = i === myId && running;
    setLabel(m.label, me ? 'أنت' : p.name, me ? '#ffe066' : p.bot ? '#e8e8e8' : '
    m.bubble.visible = p.call > 0;
    if (m.bubble.visible) m.bubble.position.y = 3.75 + Math.sin(clockT * 8) * 0.0
    m.ring.material.color.setHex(me ? 0xffe066 : TEAM_COL[p.team]);
    m.ring.scale.setScalar(me ? 1.2 : 1);
  }
  // الكورة
  const b = S.ball;
  let bx, by, bz;
  if (b.owner >= 0) {
    const o = S.players[b.owner], r = R[b.owner];
    const hold = o.gk && Math.abs(r.x) < 10.5 && Math.abs(r.z - -HL * sgn(o.team)
    const d = hold ? 0.55 : 0.8;
    bx = r.x + Math.sin(r.rot) * d; bz = r.z + Math.cos(r.rot) * d; by = hold ? 1
  } else if (client) {
    b.age = Math.min(0.3, b.age + dt);
    bx = b.x + b.vx * b.age; bz = b.z + b.vz * b.age; by = Math.max(BR, b.y + b.v
    bx = clamp(bx, -BX + BR, BX - BR); bz = clamp(bz, -BZ + BR, BZ - BR);
  } else { bx = b.x; by = b.y; bz = b.z; }
  const kb = client && b.owner < 0 ? 1 - Math.exp(-20 * dt) : 1;
  const nx = ballR.x + (bx - ballR.x) * kb, ny = ballR.y + (by - ballR.y) * kb, n
  const mdx = nx - ballR.x, mdz = nz - ballR.z, ml = Math.hypot(mdx, mdz);
  if (ml > 0.0001 && ml < 5) { tmpV.set(mdz, 0, -mdx).normalize(); ballMesh.rotat
  ballR.x = nx; ballR.y = ny; ballR.z = nz;
  ballMesh.position.set(nx, ny, nz);
  ballShadow.position.set(nx, 0.025, nz);
  const hs = clamp(1 - (ny - BR) * 0.12, 0.4, 1); ballShadow.scale.setScalar(hs);
  updateCamera(dt);
}
function updateCamera(dt) {
  let tx, tz, s = 1;
  if (running && myId >= 0) {
    const r = R[myId]; tx = r.x; tz = r.z; s = sgn(S.players[myId].team);
    tx = tx * 0.85 + ballR.x * 0.15;
    const dx = tx, dz = tz - 13.5 * s;
    setCam(dx, 10, dz, tx, 0.8, tz + 7 * s, dt);
```

```
  } else {
    cam.orbit += dt * 0.08;
    const R0 = 46;
    setCam(Math.sin(cam.orbit) * R0, 22, Math.cos(cam.orbit) * R0, ballR.x * 0.3,
  }
}
function setCam(x, y, z, lx, ly, lz, dt) {
  const k = cam.snap ? 1 : 1 - Math.exp(-6 * dt);
  cam.snap = false;
  cam.x += (x - cam.x) * k; cam.y += (y - cam.y) * k; cam.z += (z - cam.z) * k;
  cam.lx += (lx - cam.lx) * k; cam.ly += (ly - cam.ly) * k; cam.lz += (lz - cam.l
  camera.position.set(cam.x, cam.y, cam.z);
  camera.lookAt(cam.lx, cam.ly, cam.lz);
}
function resize() {
  const w = window.innerWidth, h = window.innerHeight;
  renderer.setSize(w, h, false);
  camera.aspect = w / h;
  camera.fov = w / h < 1.2 ? 70 : 58;
  camera.updateProjectionMatrix();
}
window.addEventListener('resize', resize);
window.addEventListener('orientationchange', () => setTimeout(resize, 300));

/* ================== التحكم ================== */
const joy = { id: null, ox: 0, oy: 0, x: 0, y: 0 };
const keys = {};
let sprintBtn = 0;
const joyb = $('joyb'), joyk = $('joyk'), zone = $('joyzone');
const JR = 58;
zone.addEventListener('pointerdown', (e) => {
  if (joy.id !== null) return;
  e.preventDefault(); SND.init();
  joy.id = e.pointerId; joy.ox = e.clientX; joy.oy = e.clientY; joy.x = joy.y = 0
  try { zone.setPointerCapture(e.pointerId); } catch (x) {}
  const zr = zone.getBoundingClientRect();
  joyb.style.left = (e.clientX - zr.left) + 'px'; joyb.style.top = (e.clientY - z
  joyk.style.transform = 'translate(0px,0px)';
});
zone.addEventListener('pointermove', (e) => {
  if (e.pointerId !== joy.id) return;
  e.preventDefault();
  let dx = e.clientX - joy.ox, dy = e.clientY - joy.oy;
  const l = Math.hypot(dx, dy);
  if (l > JR) { dx = (dx / l) * JR; dy = (dy / l) * JR; }
  joy.x = dx / JR; joy.y = -dy / JR;
```

```
  joyk.style.transform = `translate(${dx}px,${dy}px)`;
});
const joyEnd = (e) => {
  if (e.pointerId !== joy.id) return;
  joy.id = null; joy.x = joy.y = 0;
  joyk.style.transform = 'translate(0px,0px)'; joyb.classList.remove('act');
  joyb.style.left = ''; joyb.style.top = '';
};
zone.addEventListener('pointerup', joyEnd);
zone.addEventListener('pointercancel', joyEnd);

function bindBtn(id, down, up) {
  const el = $(id);
  el.addEventListener('pointerdown', (e) => { e.preventDefault(); SND.init(); try
  const end = (e) => { if (!el.classList.contains('down')) return; el.classList.r
  el.addEventListener('pointerup', end); el.addEventListener('pointercancel', end
}
bindBtn('bShoot', () => { LI.sh = 1; }, () => { LI.sh = 0; });
bindBtn('bPass', () => { LI.pa++; });
bindBtn('bSlide', () => { LI.sl++; });
bindBtn('bSprint', () => { sprintBtn = 1; }, () => { sprintBtn = 0; });
bindBtn('bCall', () => { LI.ca++; });

window.addEventListener('keydown', (e) => {
  if (document.activeElement && document.activeElement.tagName === 'INPUT') retur
  const k = e.key.toLowerCase();
  if (!keys[k]) {
    if (k === 'e' || k === 'k') LI.pa++;
    if (k === 'q' || k === 'l') LI.sl++;
    if (k === 'f') LI.ca++;
  }
  keys[k] = true;
  if (k === ' ') { LI.sh = 1; e.preventDefault(); }
});
window.addEventListener('keyup', (e) => {
  const k = e.key.toLowerCase(); keys[k] = false;
  if (k === ' ') LI.sh = 0;
});
document.addEventListener('touchmove', (e) => { if (running) e.preventDefault();
document.addEventListener('gesturestart', (e) => e.preventDefault());

function readInput() {
  let jx = joy.x + ((keys.d || keys.arrowright) ? 1 : 0) - ((keys.a || keys.arrow
  let jy = joy.y + ((keys.w || keys.arrowup) ? 1 : 0) - ((keys.s || keys.arrowdow
  const l = Math.hypot(jx, jy); if (l > 1) { jx /= l; jy /= l; }
  const s = myId >= 0 ? sgn(S.players[myId].team) : 1;
```

```
  LI.mx = -jx * s; LI.mz = jy * s;
  LI.sp = sprintBtn || keys.shift ? 1 : 0;
}

/* ================== الـواجهة ================== */
let toastT = 0, bigT = 0, localCharge = 0;
function status(t) { $('status').textContent = t; }
function toast(t) { const el = $('toast'); el.textContent = t; el.classList.add('
function bigText(html, color, dur) {
  const el = $('big'); el.innerHTML = html; el.style.color = color || '#fff';
  el.classList.remove('on'); void el.offsetWidth; el.classList.add('on'); bigT =
}
function playerName() {
  let n = ($('name').value || '').trim();
  if (!n) n = 'لاعب' + Math.floor(10 + Math.random() * 89);
  return n.slice(0, 14);
}
function handleEvent(e) {
  if (!running) return;
  switch (e[0]) {
    case 'k': SND.kick(e[1]); break;
    case 'w': SND.whistle(!!e[1]); break;
    case 's': SND.whoosh(); break;
    case 't': SND.thud(); if (e[1] === myId) toast('طيّحوك! 😵'); break;
    case 'c': SND.pop(); break;
    case 'g': {
      SND.goal();
      const mine = myId >= 0 && S.players[myId].team === e[1];
      bigText('قووووول! ⚽<small>' + escapeHtml(e[2] || '') + '</small>', TEAM_CS
      if (myId >= 0) toast(mine ? 'دخل فيكم هدف 🔥' : 'هدف لفريقك 😬');
      break;
    }
    case 'e': showEnd(); SND.whistle(true); break;
    case 'r': $('endbox').classList.add('hide'); bigText('مباراة جديدة!', '#ffe06
  }
}
function escapeHtml(s) { return String(s).replace(/[&<>"']/g, (c) => ({ '&': '&am
function showEnd() {
  const [a, b] = S.score;
  let t;
  if (a === b) t = 'تعادل 🤝';
  else {
    const w = a > b ? 0 : 1;
    t = myId >= 0 ? (S.players[myId].team === w ? 'خسرت 😔' : 'فزت! 🏆') : 'فاز '
  }
  $('endT').textContent = t;
```

```
  $('endS').innerHTML = `<span style="color:${TEAM_CSS[0]}">${TEAM_NAME[0]} ${a}<
  const canRestart = NET.mode !== 'client';
  $('bAgain').classList.toggle('hide', !canRestart || NET.pub);
  if (NET.pub) $('endS').innerHTML += '<br><small>شوي بعد جديدة مباراة تبدأ…</sma
  else if (!canRestart) $('endS').innerHTML += '<br><small>غرفة يبدأ مباراة جديدة
  $('endbox').classList.remove('hide');
}
function fmtTime(t) { t = Math.max(0, Math.ceil(t)); return Math.floor(t / 60) +
let hudT = 0, lastPhase = '';
function updateHUD(dt) {
  $('sB').textContent = S.score[0]; $('sR').textContent = S.score[1];
  $('tm').textContent = fmtTime(S.time);
  const me = myId >= 0 ? S.players[myId] : null;
  if (me) $('stamBar').firstElementChild.style.transform = `scaleX(${clamp(me.sta
  if (LI.sh) localCharge = Math.min(1, localCharge + dt * 1.25); else localCharge
  $('powBar').classList.toggle('hide', !LI.sh);
  $('powBar').firstElementChild.style.transform = `scaleX(${0.25 + 0.75 * localCh
  if (toastT > 0) { toastT -= dt; if (toastT <= 0) $('toast').classList.remove('o
  if (bigT > 0) { bigT -= dt; if (bigT <= 0) $('big').classList.remove('on'); }
  if (S.phase !== lastPhase) {
    if (S.phase === 'kickoff') bigText('استعد…', '#ffffff', 1.8);
    else if (S.phase === 'play' && lastPhase === 'kickoff') bigText('يلا! ⚽', '#f
    lastPhase = S.phase;
  }
  hudT -= dt;
  if (hudT <= 0) {
    hudT = 0.5;
    let info = '';
    if (NET.mode === 'host') {
      const n = 1 + NET.conns.length;
      info = (NET.pub ? 'غرفة ' : 'مباراة عامة ' + NET.room) + ' • لاعبين: ' + n;
    } else if (NET.mode === 'client') {
      info = (NET.pub ? 'غرفة ' : 'مباراة عامة ' + NET.room) + (NET.ping ? ' • خير
    } else info = 'ضد البوتات';
    $('roomInfo').textContent = info;
    $('bShare').classList.toggle('hide', !(NET.mode === 'host' && !NET.pub));
    $('bSlide').classList.toggle('cd', !!(me && me.slideCd > 0));
  }
}

function startGame() {
  running = true; cam.snap = true; lastPhase = '';
  document.body.classList.add('playing');
  $('menu').classList.add('hide'); $('hud').classList.remove('hide'); $('endbox')
  LI.sh = 0; sprintBtn = 0;
  SND.init();
```

```
  try { const el = document.documentElement; if (el.requestFullscreen && !documen
  try { if (screen.orientation && screen.orientation.lock) screen.orientation.loc
  const t = S.players[myId].team;
  toast('أنت بالفريق ' + TEAM_NAME[t] + ' • رقمك ' + S.players[myId].num);
  if (NET.mode === 'host' && !NET.pub) setTimeout(() => toast('رمز غرفتك: ' + NET
  try { localStorage.setItem('kora_name', $('name').value || ''); } catch (e) {}
}
function backToMenu(msg) {
  running = false; netClose();
  document.body.classList.remove('playing');
  $('menu').classList.remove('hide'); $('hud').classList.add('hide');
  myId = -1; S = newState(); S.autoRestart = true; placeKickoff(); cam.snap = tru
  status(msg || '');
}

$('bBots').onclick = () => {
  if (busy) return;
  SND.init(); netClose(); S = newState(); claimLocal(); placeKickoff(); startGame
};
$('bQuick').onclick = () => { SND.init(); quickPlay(); };
$('bHost').onclick = () => { SND.init(); hostRoom(); };
$('bJoin').onclick = () => { SND.init(); joinRoom($('code').value); };
$('bExit').onclick = () => backToMenu('');
$('bMenu2').onclick = () => backToMenu('');
$('bAgain').onclick = () => { if (NET.mode !== 'client') { restartMatch(); } };
$('bShare').onclick = async () => {
  const url = location.origin + location.pathname + '?room=' + NET.room;
  const text = 'تعال العب معي كورة الصحراء! رمز الغرفة: ' + NET.room;
  try {
    if (navigator.share) { await navigator.share({ title: 'كورة الصحراء', text, u
  } catch (e) { return; }
  try { await navigator.clipboard.writeText(text + '\n' + url); toast('نسخ الرابط
};

/* ================== الحلقة الرئيسية ================== */
let last = performance.now(), acc = 0;
function flushEvents() {
  for (const e of S.ev) { handleEvent(e); if (NET.mode === 'host') NET.netEv.push
  S.ev.length = 0;
}
function frame(now) {
  requestAnimationFrame(frame);
  const dt = Math.min(0.1, Math.max(0, (now - last) / 1000)); last = now;
  if (running) readInput();
  if (NET.mode !== 'client') {
    acc += dt; const h = 1 / 60; let n = 0;
```

```
    while (acc >= h && n < 8) { step(h); acc -= h; n++; }
    if (n >= 8) acc = 0;
    flushEvents();
    if (NET.mode === 'host') { NET.sendT += dt; if (NET.sendT >= 0.05) { NET.send
  } else {
    NET.inT += dt; if (NET.inT >= 0.05) { NET.inT = 0; sendInput(); }
    NET.pingT -= dt;
    if (NET.pingT <= 0) { NET.pingT = 2; try { if (NET.hostConn && NET.hostConn.o
  }
  render(dt);
  if (running) updateHUD(dt);
  renderer.render(scene, camera);
}

/* ================== البداية ================== */
try { const n = localStorage.getItem('kora_name'); if (n) $('name').value = n; }
const qRoom = new URLSearchParams(location.search).get('room');
if (qRoom) { $('code').value = qRoom.slice(0, 4); status('ط «دخول» عشان تـدخل غرفـة
S = newState(); S.autoRestart = true; placeKickoff();
buildWorld(); resize();
try { document.fonts.ready.then(() => { for (const m of MESH) m.label.userData.tx
requestAnimationFrame(frame);
window.__kora = { get S() { return S; }, NET, get myId() { return myId; }, LI, st
})();
```